Non linear precoding for quantization

Thomas Feys

June 24, 2024

1 System model

We consider a downlink massive MIMO system, the received signal vector can be written as

$$\mathbf{r} = \mathbf{H}^{\intercal}\mathbf{y} + \mathbf{v} \tag{1}$$

where $\mathbf{r} \in \mathbb{C}^K$ is the received signal vector, $\mathbf{H} \in \mathbb{C}^{M \times K}$ the channel matrix, $\mathbf{y} \in \mathcal{X}^M$ the precoded and quantized transmit symbols and $\mathbf{v} \in \mathbb{C}^K$ addative white Gaussian noise.

2 Precoding

2.1 Linear precoding

When linear precoding is considered we have

$$\mathbf{x} = \mathbf{W}\mathbf{s} \tag{2}$$

where $\mathbf{x} \in \mathbb{C}^M$ is the precoded vector, $\mathbf{W} \in \mathbb{C}^{M \times K}$ is the precoding matrix and $\mathbf{s} \in \mathbb{C}^K$ is the symbol vector. After precoding, quantization is applied as

$$\mathbf{y} = \mathcal{Q}(\mathbf{x}) = \Big[\mathcal{Q}(\Re\{x_0\}) + \jmath\mathcal{Q}(\Im\{x_0\}), \ \cdots, \ \mathcal{Q}(\Re\{x_{M-1}\}) + \jmath\mathcal{Q}(\Im\{x_{M-1}\})\Big]^{\mathsf{T}}. \tag{3}$$

The transmit vector $\mathbf{y}$ is designed under an average power constraint

$$\mathbb{E}\left(\|\mathbf{y}\|_2^2\right) \leq P_T \tag{4}$$

For linear precoding this can be linked to a normalization of the precoding matrix (although more diffuclt given the quantizer function, but doable I think using the Bussgang decomposition, see [1]).

2.2 Non linear precoding

When considering non linear precoding we have the following

$$\mathbf{y} = f\left(\mathbf{H}, \mathbf{s}\right) \tag{5}$$

where $f(\cdot,\cdot) : \mathbb{C}^{M \times K} \times \mathbb{C}^K \mapsto \mathcal{X}^M$ directly maps the channel matrix and symbol vector to quantized precoded values, where $\mathcal{X} = \mathcal{L} \times \mathcal{L}$ is the set of of output levels per antenna, with $\mathcal{L}$ being the set of real valued output levels of the DAC. The question is then how to enforce the average power constraint on $\mathbf{y}$ given that the distribution of $\mathbf{y}$ is unknown.

2.2.1 One bit case

If we consider the one bit case we have that the magnitude of each output level is equal. Hence, we can simply enforce the power constraint with equality

$\mathbb{E}\left(\|\mathbf{y}\|_2^2\right) = \|\mathbf{y}\|_2^2 = P_T$, by defining the set of output levels as $\mathcal{L} = \{\pm\sqrt{\frac{P_T}{2M}}\}$, this gives us the set of possible values at each antenna as $\mathcal{X} = \{\sqrt{\frac{P_T}{2M}}(\pm 1 \pm \jmath)\}$.

2.2.2 Multi bit case

When we consider multiple bits, the magnitude of each output level is no longer equal, hence we can not easily enforce the average power constraint. The constraint can be enforced given knowledge of the distribution of $\mathbf{y}$. However, given that $\mathbf{y}$ is the output of a neural network we don't know how it is distributed. One way to enforce this constraint would be to add a penalty term to the loss function which tries to satisfy the constraint

$$\lambda\left(\mathbb{E}\left(\|\mathbf{y}\|_2^2\right) - P_T\right) \tag{6}$$

where the expectation can be computed over the batchsize. However λ would need to be tuned manually. Another option is to rewrite the constrained optimization problem using a lagrangian, this would lead to a similar penalty term in the loss, however there is a paper [2] that uses an additional multipliers network to compute this λ.

3 Supervised Learning

As a sanity check we can do supervised learning first. (could also be nice to see how close we are to the optimal solution) For this we will consider the 1 bit DAC case, this simplifies the power normalization. We will consider $M = 8$ transmit antennas and $K = 1$ user. Considering a DAC per real and imaginary part of each antenna we have $2M = 16$ output levels to select.

Each output level can take 2 possible values, namely $\mathcal{L} = \{\pm\sqrt{\frac{P_T}{2M}}\}$, this gives us a search space of $2^{2M} = 65536$ possible combinations.

3.1 How to select the best output levels

For the single user case we can write the system model as

$$r = \mathbf{h}^\intercal \mathbf{y} + v \tag{7}$$

$$r = \mathbf{h}^\intercal f(\mathbf{h}, s) + v \tag{8}$$

Using the Bussgang decomposition this can be written as

$$r = Gs + d + v \tag{9}$$

here s, d and v are uncorrelated such that the SNR is

$$\text{SNR} = \frac{|G|^2 \sigma_s^2}{\sigma_d^2 + \sigma_v^2} \tag{10}$$

with

$$\sigma_s^2 = \mathbb{E}\left(|s|^2\right) = 1 \tag{11}$$

$$\sigma_v^2 = \mathbb{E}\left(|v|^2\right) \tag{12}$$

$$\sigma_d^2 = \mathbb{E}\left(|r|^2\right) - |G|^2 \sigma_s^2 - \sigma_v^2 \tag{13}$$

$$G = \frac{\mathbb{E}\left(rs^*\right)}{\sigma_s^2}. \tag{14}$$

The problem with this is the following, let's say we want to compute the optimal output levels for channel realization $\mathbf{h}_0$ and symbol s_0, we cannot compute the above expectationns given that we only have a single symbol.

Another way to do this would be to simply take the output levels that produce the smalles MSE between the received and transmitted symbols. In

this case we can assume that the receiver performs equalization as

$$\hat{s} = \beta r. \tag{15}$$

Then we select the output levels that minimize

$$\min \|s - \beta r\|_2^2 \tag{16}$$

this is equivalent to

$$\min_{\mathbf{y} \in \mathcal{X}^M, \beta \in \mathbb{R}^+} \|s - \beta \mathbf{h}^\intercal \mathbf{y}\|_2^2 + \beta^2 \sigma_v^2 \tag{17}$$

The question then is how do we get β, this should be computed once per channel, not once per symbol! Some ideas can be found here [3].

4 Unsupervised Learning

Let's consider how a neural network could learn non linear precoding in an unsipervised manner. The neural network would take as an input the channel matrix $\mathbf{H}$ and the user symbol vector $\mathbf{s}$. It's output would be a probability vector for the real and imaginary part of each antenna

$$\mathbf{p} = f(\mathbf{H}, \mathbf{s}; \boldsymbol{\theta}) \tag{18}$$

$$= [\mathbf{p}_{\Re,0}, \mathbf{p}_{\Im,0}, \cdots, \mathbf{p}_{\Re,M-1}, \mathbf{p}_{\Im,M-1}]^\intercal \tag{19}$$

where $\mathbf{p} \in [0,1]^{2M|\mathcal{L}|}$ and $\mathbf{p}_{\Re,m}, \mathbf{p}_{\Im,m} \in [0,1]^{|\mathcal{L}|}$. $\mathbf{p}_{\Re,m}, \mathbf{p}_{\Im,m}$ represent the probability vector for the real and imaginary parts of the DAC output for antenna m, each of these vectors is a result of a softmax activation, which can be interpreted as the parameters of a categorical distribution. The softmax

of a vector $\mathbf{a}$ is computed as

$$\text{softmax}(a_i) = \frac{e^{a_i}}{\sum_j e^{a_j}}. \tag{20}$$

If we know the correct labels for $\mathbf{p}_{\Re,m}, \mathbf{p}_{\Im,m}$ we could simply learn them in a supervised manner by minimizing the cross entropy loss between the output of the softmax layers and the labels.

Given that we want to train in an unsupervised manner we want to interprete the output of the softmax layer as the probability each output level has to be the 'best' value. To do so we would like to take the argmax of these probability vectors to select the maximum likelihood output levels. As an example let's do this for a single probability vector

$$\mathbf{z} = \text{onehot}\left(\arg\max_i \left(\mathbf{p}_{\Re,m}\right)\right). \tag{21}$$

The problem here is that the argmax function is non differentiable. Hence we can not sample from this categorical distribution in a way that allows us to backpropagate through it. However, we can approximate sampling from this categorical distribution in a differentiable way using the Gumbel-softmax trick [4, 5].

Sampling of the categorical distribution can be rewriten as

$$\mathbf{z} = \text{onehot}\left(\arg\max_i \left(\mathbf{g} + \log(\mathbf{p}_{\Re,m})\right)\right) \tag{22}$$

where $\mathbf{g} = [g_0, \cdots, g_{|\mathcal{L}|-1}]^\intercal$ is a vector with samples drawn from the standard Gumbel distribution $g_i \sim \text{Gumbel}(0, 1)$. This reparametrization, recasts the stochastic sampling process into a combination of a deterministic function which depends on the parameters of the categorical distribution and stochastic element which is the sampling of some independent noise drawn from the

fixed Gumbel distribution. We do this so that we can compute the gradients of the diterministic function, and as such the gradients of the parameters of the categorical distribution. In simple terms this allows us to optimize the output of the softmax layers/ the parameters of the categorical distribution. However we still have the problem of the argmax function. We can use the softmax function instead of the argmax function in order to have a differentiable approximation of the argmax function. Hence in the end this gives us

$$z_i = \frac{e^{\frac{g_i + \log([\mathbf{p}_{\Re,m}]_i)}{\tau}}}{\sum_j e^{\frac{g_j + \log([\mathbf{p}_{\Re,m}]_j)}{\tau}}} \tag{23}$$

note that a temperature parameter τ has been added to the softmax approximation in order to control the degree of approximation. For $\tau \to 0$ the softmax approaches the argmax function, as $\tau \to \infty$ the sample vector $\mathbf{z}$ becomes uniformly distributed.

Note that this continuous approximation means that continuous vectors are being used during training rather then discretized one-hot vectors. This is ok in some systems but might not be ideal in our scenario since we are considering quantization. Hence if we want discrete outputs we can use the straight through Gumbel-Softmax, here we discretize the continuous values in the forward pass by using the argmax. However, we still use the Gumbel-Softmax sample in the backward pass to approximate the gradients. (this is supported in pytorch)

Once we have our one hot vector we can simply multiply it elementwise with the vector of possible output levels (let's call this vector $\mathbf{l}$). This then

gives us the real part of the precoded symbol at antenna m as

$$\Re\left(y_m\right) = \mathbf{z} \odot \mathbf{l}. \tag{24}$$

In this way we can construct the full precoded vector $\mathbf{y}$ and then use it to maximize an unsupervised loss function such as the sumrate.

5 Neural Net Architecture

5.1 Properties to Follow

Which NN architecture do we use to perform this non-linear percoding? In general we have the following precoding function

$$\mathbf{x} = f(\mathbf{H}, \mathbf{s}) \tag{25}$$

where $\mathbf{H} \in \mathbb{C}^{M \times K}, \mathbf{s} \in \mathbb{C}^{K \times 1}$ and $\mathbf{x} \in \mathcal{X}^{M \times 1}$. We want to maintain the following properties

- If the rows of $\mathbf{H}$ are permuted, we want the rows of $\mathbf{x}$ to be permuted accordingly $\Rightarrow f(\mathbf{\Pi H}, \mathbf{s}) = \mathbf{\Pi x}$, where $\mathbf{\Pi}$ is a permutation matrix. Hence the GNN/function should be permutation equivariant with respect to the rows of the channel matrix.

- If the columns of $\mathbf{H}$ are permuted, we want the rows of $\mathbf{x}$ to remain unchanged $\Rightarrow f(\mathbf{H\Pi}, \mathbf{s}) = \mathbf{x}$. Hence the GNN/function should be permutation invariant with respect to the columns of the channel matrix.

- If the rows (= elements cause just a vector) of $\mathbf{s}$ are permuted, we want the rows of $\mathbf{x}$ to remain unchanged $\Rightarrow f(\mathbf{H}, \mathbf{\Pi s}) = \mathbf{x}$. Hence

the GNN/function should be perumtation invariant with respect to the rows of the symbol vector.

These properties should be encoded in the NN architecture. TODO It is actually not so decoupled, if we permute the columns of the channel matrix, we should also permute the rows of the symbol vector accordingly in order to maintain concitency. Let's reformulate

- If we permute the user ordering with permutation matrix $\mathbf{\Pi} \in \{0,1\}^{K\times K}$, we have $\mathbf{H\Pi}$, $\mathbf{\Pi s}$. In this scenario we want the output of the precoding function to remain unchanged, hence $f(\mathbf{H\Pi}, \mathbf{\Pi s}) = \mathbf{x}$. The learned function should thus be permutation invariant with respect to the symbol vector and columns of the channel matrix.

- if we permute the antenna ordering with permutation matrix $\mathbf{\Pi} \in \{0,1\}^{M\times M}$, we have $\mathbf{\Pi H}$. In this scenario we want the output of the precoding function to be permuted accordingly. Hence, we have $f(\mathbf{\Pi H}, \mathbf{s}) = \mathbf{\Pi x}$. The learned function should thus be permutation equivariant with respect to the rows of the channel matrix.

5.2 Generalized message passing

Here we describe generalized message passing for GNNs but we leave out the use of graph level features. One layer of this message passing is defined as

$$\mathbf{z}_{(u,v)}^{(l)} = \text{update}_{\text{edge}}\left(\mathbf{z}_{(u,v)}^{(l-1)}, \mathbf{z}_u^{(l-1)}, \mathbf{z}_v^{(l-1)}\right) \tag{26}$$

$$\mathbf{m}_{\mathcal{N}(u)} = \text{aggregate}_{\text{node}}\left(\{\mathbf{z}_{(u,v)}^{(l)}, \forall v \in \mathcal{N}(u)\}\right) \tag{27}$$

$$\mathbf{z}_u^{(l)} = \text{update}_{\text{node}}\left(\mathbf{z}_u^{(l-1)}, \mathbf{m}_{\mathcal{N}(u)}\right) \tag{28}$$

here u, v denote nodes while (u, v) denotes the edge between u and v. The update and aggregate functions can be selected as desired, a straighforward implementation is given below

$$\mathbf{z}_{(u,v)}^{(l)} = \sigma\left(\mathbf{W}_{\text{edge}}\mathbf{z}_{(u,v)}^{(l-1)}, \mathbf{W}_u\mathbf{z}_u^{(l-1)}, \mathbf{W}_v\mathbf{z}_v^{(l-1)}\right) \tag{29}$$

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{1}{|\mathcal{N}(u)|}\sum_{v\in\mathcal{N}(u)} \mathbf{z}_{(u,v)}^{(l)} \tag{30}$$

$$\mathbf{z}_u^{(l)} = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{z}_u^{(l-1)} + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(u)}\right). \tag{31}$$

For the specific case of MIMO nonlinear precoding this can be written as

$$\mathbf{z}_{(m,k)}^{(l)} = \sigma\left(\mathbf{W}_{\text{edge}}\mathbf{z}_{(m,k)}^{(l-1)} + \mathbf{W}_m\mathbf{z}_m^{(l-1)} + \mathbf{W}_k\mathbf{z}_k^{(l-1)}\right) \tag{32}$$

$$\mathbf{m}_{\mathcal{N}(k)} = \frac{1}{|\mathcal{N}(k)|}\sum_{m'\in\mathcal{N}(k)} \mathbf{z}_{(m',k)}^{(l)} \tag{33}$$

$$\mathbf{m}_{\mathcal{N}(m)} = \frac{1}{|\mathcal{N}(m)|}\sum_{k'\in\mathcal{N}(m)} \mathbf{z}_{(m,k')}^{(l)} \tag{34}$$

$$\mathbf{z}_m^{(l)} = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{z}_m^{(l-1)} + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(m)}\right) \tag{35}$$

$$\mathbf{z}_k^{(l)} = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{z}_k^{(l-1)} + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(k)}\right). \tag{36}$$

We could make this more general by defining seperate aggregate and node update functions for the user and antenna nodes

References

[1] J. Choi, J. Park, and N. Lee, "Energy efficiency maximization precoding for quantized massive mimo systems," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 6803–6817, 2022.

[2] C. Sun, D. Liu, and C. Yang, "Model-free unsupervised learning for optimization problems with constraints," in *2019 25th Asia-Pacific Conference on Communications (APCC)*, 2019, pp. 392–397.

[3] S. Jacobsson, G. Durisi, M. Coldrey, T. Goldstein, and C. Studer, "Nonlinear 1-bit precoding for massive mu-mimo with higher-order modulation," 2016.

[4] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," 2017.

[5] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," 2017.